

Autoruns y binarios de arranque

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Windows ejecuta programas al inicio o al login desde varias claves de registro (Run, RunOnce) y carpetas de startup, a veces con privilegios altos. Si el binario referenciado es escribible por tu usuario, o la clave de registro es modificable, colocas tu ejecutable y corre con los privilegios del que dispara el autorun. Variante del winprivesc-modelo.

El software de terceros registra autoruns sin proteger bien los permisos del binario o de la clave. Si un admin inicia sesión (o el arranque es como SYSTEM), tu binario reemplazado se ejecuta con ese contexto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En la enumeración (enum-privesc-windows). WinPEAS y autorunsc listan los autoruns y marcan los que tienen permisos débiles.

Cómo identificarla

- Un binario referenciado en una clave Run es escribible por tu usuario (accesschk).
- Una clave de registro de autorun modificable por ti.
- autorunsc marca un autorun con permisos flojos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

accesschk sobre el binario referenciado por cada entrada de Run/RunOnce.

Evidencia esperada: Un permiso de escritura (FILE_WRITE_DATA/W) para tu usuario o grupo sobre el .exe de autorun.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
```

Objetivo y contexto: Lista los programas que arrancan con el sistema desde la clave Run de maquina. Son candidatos si el binario o la clave son escribibles.

Resultado esperado: Nombres y rutas de los binarios de autorun. Para cada uno, comprueba con accesschk si puedes escribir el .exe o modificar la clave.

Capa 2: prueba documentada

```
accesschk.exe /accepteula -wvu "C:\ruta\autorun.exe"
```

Objetivo y contexto: Comprueba si tu usuario tiene permiso de escritura sobre el binario de autorun. Esa escritura es lo que te deja reemplazarlo por tu payload.

Resultado esperado: Permisos por usuario/grupo. Un FILE_WRITE_DATA/W para tu usuario o un grupo tuyo = puedes reemplazar el .exe. Si no, prueba modificar la clave de registro.

Anatomía de los payloads

La primera prueba es `reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un permiso de escritura (FILE_WRITE_DATA/W) para tu usuario o grupo sobre el .exe de autorun.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera autoruns (registro Run/RunOnce y carpetas startup).
- Comprueba permisos del binario y de la clave (accesschk).
- Reemplaza el binario escribible por tu payload (o apunta la clave al tuyo).
- Espera el proximo login/arranque del usuario privilegiado y recoge la ejecucion.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ningun autorun referencia un binario escribible por tu usuario, y las claves de registro no son modificables.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si un programa que arranca automáticamente con privilegios apunta a un binario que puedes escribir, lo reemplazas. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (registry-autoruns)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.